Industry Project: Manufacturing Defect Detection Using Transfer Learning

Business Problem

Imagine you work as a Computer Vision Engineer at an automobile manufacturing company.

Every day, thousands of metal parts are produced. Inspecting each part manually is:

- Slow
- Expensive
- Error-prone

Your task is to build an AI system that automatically detects defective parts from images.

This is exactly the type of project companies like Bosch, Siemens, Tesla, BMW, and Samsung implement.

Problem Statement

Goal

Build a deep learning model that classifies images into:

- Defective
- Non-Defective

using Transfer Learning.

Why this project?

In industry:

Nobody trains CNNs from scratch.

Instead, companies use

- ResNet50
- EfficientNet
- MobileNet
- DenseNet
- Vision Transformers

pretrained on ImageNet and fine-tune them.

Dataset

Use the NEU Surface Defect Database.

It contains industrial steel surface images with defects.

Classes include

- Crazing
- Inclusion
- Patches
- Pitted Surface
- Rolled-in Scale
- Scratches

This dataset is widely used in research and industrial computer vision benchmarking.

Industry Workflow

Raw Images

↓

Image Cleaning

↓

Resize Images

↓

Data Augmentation

↓

Load Pretrained Model

↓

Freeze Backbone

↓

Train Classifier

↓

Fine Tune

↓

Evaluate

↓

Deploy

This is the exact workflow followed in many production image-classification pipelines.

Project Milestones

Phase 1

Data Understanding

Tasks

- Explore dataset
- Count images
- Plot class distribution
- Display sample images

Deliverable
EDA report

Phase 2

Data Preprocessing
Resize
224×224
Normalize
0-255

↓

0-1
Augmentation

- Rotation
- Zoom
- Horizontal Flip
- Brightness
- Contrast

Deliverable
Training pipeline

Phase 3

Baseline CNN
Build your own CNN
Conv

↓

Pooling

↓

Conv

↓

Pooling

↓

Dense

↓

Output
Record

- Accuracy
- Precision
- Recall

- F1 Score

Phase 4

Transfer Learning
Load
ResNet50
Freeze
include_top=False
Replace
GlobalAveragePooling

↓

Dense

↓

Dropout

↓

Output
Train
Compare with baseline CNN.

Phase 5

Fine Tuning
Unfreeze
Last 20 Layers
Train again
Observe

- Accuracy increase
- Loss reduction

Phase 6

Evaluation
Create

- Confusion Matrix
- Precision
- Recall
- F1-score
- ROC Curve (binary) or One-vs-Rest ROC (multiclass)
- Precision–Recall Curve
- Misclassified image analysis

Phase 7

Model Explainability
Generate Grad-CAM heatmaps.

Answer questions like:

- Why did the model predict "Scratch"?
- Which image regions influenced the decision?

This is widely expected in regulated industries.

Phase 8

Deployment
Build a simple web app using Flask or FastAPI.
Features:

- Upload an image
- Predict defect class
- Display confidence score
- Show Grad-CAM heatmap
- Keep a prediction log

Model Comparison

Train and compare:

Model	Train Time	Accuracy	Parameters	Notes
Custom CNN				Baseline
MobileNetV2				Lightweight
ResNet50				Strong baseline
EfficientNetB0				High accuracy
DenseNet121				Feature reuse

Hyperparameter Tuning

Experiment with:

- Learning Rate
- Batch Size
- Epochs
- Optimizer (Adam, SGD)
- Dropout
- Number of unfrozen layers
- Image size (224 vs. 299 if using models that benefit from it)

Industry Documentation

Create:

README

Include:

- Problem Statement
- Dataset
- Workflow
- Results
- Screenshots
- Deployment Instructions

Technical Report

Include:

- Business Objective
- Data Understanding
- Model Architecture
- Transfer Learning Strategy
- Fine-Tuning Strategy
- Metrics
- Challenges
- Future Work

GitHub Repository Structure

transfer-learning-defect-detection/

```
│
├─── data/
├─── notebooks/
│    ├─── 01_EDA.ipynb
│    ├─── 02_Preprocessing.ipynb
│    ├─── 03_Baseline_CNN.ipynb
│    ├─── 04_Transfer_Learning.ipynb
│    ├─── 05_Fine_Tuning.ipynb
│    └─── 06_GradCAM.ipynb
│
├─── models/
├─── app/
├─── templates/
├─── static/
├─── reports/
├─── requirements.txt
└─── README.md
```

Skills Gain

By completing this project, you'll gain hands-on experience with:

- Image preprocessing and augmentation
- Transfer learning using TensorFlow/Keras
- Feature extraction vs. fine-tuning
- Model evaluation and comparison
- Hyperparameter tuning
- Explainable AI (Grad-CAM)
- Model deployment with Flask/FastAPI
- Project organization and documentation

These are all skills commonly expected for entry-level and mid-level Computer Vision and Deep Learning roles.

Recommendation for how we build this

Rather than jumping straight to the final code, we should build it the way an industry team would:

1. Obtain and inspect the dataset.

2. Perform EDA and preprocessing.
3. Build and evaluate a baseline CNN.
4. Implement transfer learning with ResNet50.
5. Fine-tune the model and compare results.
6. Add Grad-CAM for explainability.
7. Deploy the model as a web application.
8. Package the project with proper documentation and GitHub structure.

PROJECT SUBMISSION LINK:

https://docs.google.com/forms/d/e/1FAIpQLSfK5xiGgbxcGiANFbEXdxdkZ2JmiQ88gSngExZRxcEfP8KHWg/viewform?usp=header